

1 Algoritmos - Actividad Guiada 1

Nombre: Luis Chumpitaz Santivañez

Repositorio en GitHub - Luis Chumpitaz Santivanez

1.1 Torres de Hanoi con Divide y vencerás

```
[5]: def Torres_Hanoi(N, desde, hasta):  
    if N ==1 :  
        print("Lleva la ficha " ,desde , " hasta " , hasta )  
  
    else:  
        #Torres_Hanoi(N-1, desde, 6-desde-hasta )  
        Torres_Hanoi(N-1, desde, 6-desde-hasta )  
        print("Lleva la ficha " ,desde , " hasta " , hasta )  
        #Torres_Hanoi(N-1,6-desde-hasta, hasta )  
        Torres_Hanoi(N-1, 6-desde-hasta , hasta )
```

```
[6]: Torres_Hanoi(3, 1 , 3)
```

```
Lleva la ficha 1 hasta 3  
Lleva la ficha 1 hasta 2  
Lleva la ficha 3 hasta 2  
Lleva la ficha 1 hasta 3  
Lleva la ficha 2 hasta 1  
Lleva la ficha 2 hasta 3  
Lleva la ficha 1 hasta 3
```

```
[16]: #Sucesión_de_Fibonacci  
#https://es.wikipedia.org/wiki/Sucesi%C3%B3n_de_Fibonacci  
#Calculo del termino n-simo de la suscesión de Fibonacci  
def Fibonacci(N:int):  
    if N < 2:  
        return 1  
    else:  
        return Fibonacci(N-1)+Fibonacci(N-2)
```

```
[17]: print(f'Fibonacci(1) - resultado:',Fibonacci(1))
      print(f'Fibonacci(2) - resultado:',Fibonacci(2))
      print(f'Fibonacci(3) - resultado:',Fibonacci(3))
      print(f'Fibonacci(4) - resultado:',Fibonacci(4))
      print(f'Fibonacci(5) - resultado:',Fibonacci(5))
```

```
Fibonacci(1) - resultado: 1
Fibonacci(2) - resultado: 2
Fibonacci(3) - resultado: 3
Fibonacci(4) - resultado: 5
Fibonacci(5) - resultado: 8
```

1.2 Devolución de cambio por técnica voraz

```
[19]: def cambio_monedas(N, SM):
      SOLUCION = [0]*len(SM)    #SOLUCION = [0,0,0,0,..]
      ValorAcumulado = 0

      for i,valor in enumerate(SM):
          monedas = (N-ValorAcumulado)//valor
          SOLUCION[i] = monedas
          ValorAcumulado = ValorAcumulado + monedas*valor

      if ValorAcumulado == N:
          return SOLUCION
```

```
[20]: cambio_monedas(15,[25,10,5,1])
```

```
[20]: [0, 1, 1, 0]
```

1.3 N-Reinas por técnica de vuelta atrás

```
[1]: def escribe(S):
      n = len(S)
      for x in range(n):
          print("")
          for i in range(n):
              if S[i] == x+1:
                  print(" X ", end="")
              else:
                  print(" - ", end="")

      def es_prometedora(SOLUCION,etapa):
          #print(SOLUCION)
          #Si la solución tiene dos valores iguales no es valida => Dos reinas en la
          ↪misma fila
```

```

for i in range(etapa+1):
    #print("El valor " + str(SOLUCION[i]) + " está " + str(SOLUCION.
↪count(SOLUCION[i])) + " veces")
    if SOLUCION.count(SOLUCION[i]) > 1:
        return False

    #Verifica las diagonales
    for j in range(i+1, etapa +1 ):
        #print("Comprobando diagonal de " + str(i) + " y " + str(j))
        if abs(i-j) == abs(SOLUCION[i]-SOLUCION[j]) : return False
    return True

def reinas(N, solucion=[], etapa=0):
    if len(solucion) == 0:
        solucion=[0 for i in range(N)]

    for i in range(1, N+1):
        solucion[etapa] = i

        if es_prometedora(solucion, etapa):
            if etapa == N-1:
                print(solucion)
                #escribe(solucion)
                print()
            else:
                reinas(N, solucion, etapa+1)
        else:
            None

    solucion[etapa] = 0

```

```
[2]: reinas(8)
```

```
[1, 5, 8, 6, 3, 7, 2, 4]
```

```
[1, 6, 8, 3, 7, 4, 2, 5]
```

```
[1, 7, 4, 6, 8, 2, 5, 3]
```

```
[1, 7, 5, 8, 2, 4, 6, 3]
```

```
[2, 4, 6, 8, 3, 1, 7, 5]
```

```
[2, 5, 7, 1, 3, 8, 6, 4]
```

```
[2, 5, 7, 4, 1, 8, 6, 3]
```

[2, 6, 1, 7, 4, 8, 3, 5]

[2, 6, 8, 3, 1, 4, 7, 5]

[2, 7, 3, 6, 8, 5, 1, 4]

[2, 7, 5, 8, 1, 4, 6, 3]

[2, 8, 6, 1, 3, 5, 7, 4]

[3, 1, 7, 5, 8, 2, 4, 6]

[3, 5, 2, 8, 1, 7, 4, 6]

[3, 5, 2, 8, 6, 4, 7, 1]

[3, 5, 7, 1, 4, 2, 8, 6]

[3, 5, 8, 4, 1, 7, 2, 6]

[3, 6, 2, 5, 8, 1, 7, 4]

[3, 6, 2, 7, 1, 4, 8, 5]

[3, 6, 2, 7, 5, 1, 8, 4]

[3, 6, 4, 1, 8, 5, 7, 2]

[3, 6, 4, 2, 8, 5, 7, 1]

[3, 6, 8, 1, 4, 7, 5, 2]

[3, 6, 8, 1, 5, 7, 2, 4]

[3, 6, 8, 2, 4, 1, 7, 5]

[3, 7, 2, 8, 5, 1, 4, 6]

[3, 7, 2, 8, 6, 4, 1, 5]

[3, 8, 4, 7, 1, 6, 2, 5]

[4, 1, 5, 8, 2, 7, 3, 6]

[4, 1, 5, 8, 6, 3, 7, 2]

[4, 2, 5, 8, 6, 1, 3, 7]

[4, 2, 7, 3, 6, 8, 1, 5]

[4, 2, 7, 3, 6, 8, 5, 1]

[4, 2, 7, 5, 1, 8, 6, 3]

[4, 2, 8, 5, 7, 1, 3, 6]

[4, 2, 8, 6, 1, 3, 5, 7]

[4, 6, 1, 5, 2, 8, 3, 7]

[4, 6, 8, 2, 7, 1, 3, 5]

[4, 6, 8, 3, 1, 7, 5, 2]

[4, 7, 1, 8, 5, 2, 6, 3]

[4, 7, 3, 8, 2, 5, 1, 6]

[4, 7, 5, 2, 6, 1, 3, 8]

[4, 7, 5, 3, 1, 6, 8, 2]

[4, 8, 1, 3, 6, 2, 7, 5]

[4, 8, 1, 5, 7, 2, 6, 3]

[4, 8, 5, 3, 1, 7, 2, 6]

[5, 1, 4, 6, 8, 2, 7, 3]

[5, 1, 8, 4, 2, 7, 3, 6]

[5, 1, 8, 6, 3, 7, 2, 4]

[5, 2, 4, 6, 8, 3, 1, 7]

[5, 2, 4, 7, 3, 8, 6, 1]

[5, 2, 6, 1, 7, 4, 8, 3]

[5, 2, 8, 1, 4, 7, 3, 6]

[5, 3, 1, 6, 8, 2, 4, 7]

[5, 3, 1, 7, 2, 8, 6, 4]

[5, 3, 8, 4, 7, 1, 6, 2]

[5, 7, 1, 3, 8, 6, 4, 2]

[5, 7, 1, 4, 2, 8, 6, 3]

[5, 7, 2, 4, 8, 1, 3, 6]

[5, 7, 2, 6, 3, 1, 4, 8]

[5, 7, 2, 6, 3, 1, 8, 4]

[5, 7, 4, 1, 3, 8, 6, 2]

[5, 8, 4, 1, 3, 6, 2, 7]

[5, 8, 4, 1, 7, 2, 6, 3]

[6, 1, 5, 2, 8, 3, 7, 4]

[6, 2, 7, 1, 3, 5, 8, 4]

[6, 2, 7, 1, 4, 8, 5, 3]

[6, 3, 1, 7, 5, 8, 2, 4]

[6, 3, 1, 8, 4, 2, 7, 5]

[6, 3, 1, 8, 5, 2, 4, 7]

[6, 3, 5, 7, 1, 4, 2, 8]

[6, 3, 5, 8, 1, 4, 2, 7]

[6, 3, 7, 2, 4, 8, 1, 5]

[6, 3, 7, 2, 8, 5, 1, 4]

[6, 3, 7, 4, 1, 8, 2, 5]

[6, 4, 1, 5, 8, 2, 7, 3]

[6, 4, 2, 8, 5, 7, 1, 3]

[6, 4, 7, 1, 3, 5, 2, 8]

[6, 4, 7, 1, 8, 2, 5, 3]

```
[6, 8, 2, 4, 1, 7, 5, 3]

[7, 1, 3, 8, 6, 4, 2, 5]

[7, 2, 4, 1, 8, 5, 3, 6]

[7, 2, 6, 3, 1, 4, 8, 5]

[7, 3, 1, 6, 8, 5, 2, 4]

[7, 3, 8, 2, 5, 1, 6, 4]

[7, 4, 2, 5, 8, 1, 3, 6]

[7, 4, 2, 8, 6, 1, 3, 5]

[7, 5, 3, 1, 6, 8, 2, 4]

[8, 2, 4, 1, 7, 5, 3, 6]

[8, 2, 5, 3, 1, 7, 4, 6]

[8, 3, 1, 6, 2, 5, 7, 4]

[8, 4, 1, 3, 6, 2, 7, 5]
```

1.4 Viaje por el río. Programación dinámica

```
[29]: TARIFAS = [
    [0,5,4,3,999,999,999],
    [999,0,999,2,3,999,11],
    [999,999, 0,1,999,4,10],
    [999,999,999, 0,5,6,9],
    [999,999, 999,999,0,999,4],
    [999,999, 999,999,999,0,3],
    [999,999,999,999,999,999,0]
]

#####
def Precios(TARIFAS):
    #####
    #Total de Nodos
    N = len(TARIFAS[0])
```

```

#Iniciación de la tabla de precios
PRECIOS = [ [9999]*N for i in range(9999)*N]
RUTA = [ [""]*N for i in range(9999)*N]

for i in range(0,N-1):
    RUTA[i][i] = i #Para ir de i a i se "pasa por i"
    PRECIOS[i][i] = 0 #Para ir de i a i se se paga 0
    for j in range(i+1, N):
        MIN = TARIFAS[i][j]
        RUTA[i][j] = i

        for k in range(i, j):
            if PRECIOS[i][k] + TARIFAS[k][j] < MIN:
                MIN = min(MIN, PRECIOS[i][k] + TARIFAS[k][j] )
                RUTA[i][j] = k #Anota que para ir de i a j hay que pasar
    por k
        PRECIOS[i][j] = MIN

    return PRECIOS,RUTA
#####

PRECIOS,RUTA = Precios(TARIFAS)
#print(PRECIOS[0][6])

print("PRECIOS")
for i in range(len(TARIFAS)):
    print(PRECIOS[i])

print("\nRUTA")
for i in range(len(TARIFAS)):
    print(RUTA[i])

#Determinar la ruta con Recursividad
def calcular_ruta(RUTA, desde, hasta):
    if desde == hasta:
        #print("Ir a :" + str(desde))
        return ""
    else:
        return str(calcular_ruta( RUTA, desde, RUTA[desde][hasta])) + \
            ',' + \
            str(RUTA[desde][hasta] \
            )

print("\nLa ruta es:")
calcular_ruta(RUTA, 0,6)

```


PRECIOS

```
[0, 5, 4, 3, 8, 8, 11]
[9999, 0, 999, 2, 3, 8, 7]
[9999, 9999, 0, 1, 6, 4, 7]
[9999, 9999, 9999, 0, 5, 6, 9]
[9999, 9999, 9999, 9999, 0, 999, 4]
[9999, 9999, 9999, 9999, 9999, 0, 3]
[9999, 9999, 9999, 9999, 9999, 9999, 9999]
```

ruta

```
[0, 0, 0, 0, 1, 2, 5]
['', 1, 1, 1, 1, 3, 4]
['', '', 2, 2, 3, 2, 5]
['', '', '', 3, 3, 3, 3]
['', '', '', '', 4, 4, 4]
['', '', '', '', '', 5, 5]
['', '', '', '', '', '', '']
```

La ruta es:

[29]: ',0,2,5'

1.5 Practica Individual

Problema: Encontrar los dos puntos más cercanos * Dado un conjunto de puntos se trata de encontrar los dos puntos más cercanos. * Guia para aprendizaje: - Suponer 1D, osea, una lista de numeros

[3403 4537 9089 9746 7259 ...]

- Primer intento fuerza bruta - Calcular la complejidad. ¿Qué se puede mejorar? - Segundo intento. Aplicar Divide y Vencerás. - Calcular la complejidad. ¿Se puede mejorar? - Extender el algoritmo a 2D:

$$\begin{bmatrix} 1122 & 6175 \\ 135 & 4076 \\ 7296 & 2741 \end{bmatrix}$$

- Extender el algoritmo a 3D.

Referencia

- **Paper:** A Simple, Ecient, Parallelizable Algorithm for Approximated Nearest Neighbors, autores: Sebastian Ferrada, Benjamin Bustos, and Nora Reyes - Institute for Foundational Research on Data Department of Computer Science University of Chile Santiago, Chile
- **Codigo C++:** [The closest pair of points problem - Using Brute-Force and Divide & Conquer Strategies](#)
- **Material de clase:** Algoritmos de optimización - Módulo de Matemáticas

```
[ ]: import math
from typing import List, Tuple, Union
```

```

#
Point1D = float
Point2D = Tuple[float, float]
Point3D = Tuple[float, float, float]
Point = Union[Point1D, Point2D, Point3D]

def calcular_distancia(p1, p2):
    # Calcular distancia entre dos puntos, basado en la función "double_
    ↪calDist(Point p1, Point p2)"
    if isinstance(p1, (int, float)) and isinstance(p2, (int, float)):
        return abs(p1 - p2)
    return math.sqrt(sum((a - b) ** 2 for a, b in zip(p1, p2)))

def puntos_cercanos_fuerza_bruta(points):
    # Algoritmo de fuerza bruta, basado en la función "vector<Point>_
    ↪bruteForce(vector<Point> p)" y
    # del paper "Algorithm 2: Brute-force 1-NN similarity join."
    n = len(points)
    if n < 2:
        return float('inf'), (None, None)
    min_dist = float('inf')
    pair = (points[0], points[1])
    # comparar cada elemento con los demas
    for i in range(n):
        for j in range(i + 1, n):
            dist = calcular_distancia(points[i], points[j])
            if dist < min_dist:
                min_dist = dist
                pair = (points[i], points[j])
    return min_dist, pair

def puntos_cercanos_1D(points_ordenados):
    min_dist = float('inf')
    pair = (None, None)
    for i in range(len(points_ordenados)-1):
        dist = points_ordenados[i+1] - points_ordenados[i]
        if dist < min_dist:
            min_dist = dist
            pair = (points_ordenados[i], points_ordenados[i+1])
    return min_dist, pair

def puntos_cercanos_2D_rekursiva(px, py):
    # Función recursiva de divide y venceras en 2D
    # Basado en la función "vector<Point> closestPair(vector<Point> points)"
    n = len(px)
    if n <= 3:
        return puntos_cercanos_fuerza_bruta(px)

```

```

mid = n//2
mid_point = px[mid]

py_left = [p for p in py if]

```

```

[4]: import math

def obtener_coordenada_eje(punto, indice_eje):
    #Obtener la coordenada de un punto x, y, z
    if isinstance(punto, (int, float)):
        return punto
    return punto[indice_eje]

def calcular_distancia(p1, p2):
    # Calcular distancia entre dos puntos, basado en el codigo en
    ↪ClosessPair_Code.cpp sobre "double calDist(Point p1, Point p2)"
    # numeros simples 1D
    if isinstance(p1, (int, float)) and isinstance(p2, (int, float)):
        return abs(p1 - p2)

    # numeros en 2D o 3D
    suma_cuadrados = 0.0
    for i in range(len(p1)):
        diferencia = p1[i] - p2[i]
        suma_cuadrados = suma_cuadrados + (diferencia * diferencia)

    return math.sqrt(suma_cuadrados)

def ordenar_puntos_por_eje(lista_puntos, indice_eje):
    # Ordenar los puntos de un eje especifico x, y, z
    # Basado en el codigo en ClosessPair_Code.cpp sobre "vector<Point>
    ↪mergeSort(vector<Point> vect, string coord)"
    puntos = list(lista_puntos)
    n = len(puntos)
    for i in range(n):
        indice_minimo = i
        for j in range(i + 1, n):
            val_j = obtener_coordenada_eje(puntos[j], indice_eje)
            val_min = obtener_coordenada_eje(puntos[indice_minimo], indice_eje)
            if val_j < val_min:
                indice_minimo = j
        # Intercambio de posicion
        temp = puntos[i]
        puntos[i] = puntos[indice_minimo]
        puntos[indice_minimo] = temp
    return puntos

```

```

def filtrar_puntos_por_limite_x(lista_puntos, valor_limite, es_menor_igual):
    # Separacion de puntos
    resultado = []
    for p in lista_puntos:
        coord_x = obtener_coordenada_eje(p, 0)
        if es_menor_igual:
            if coord_x <= valor_limite:
                resultado.append(p)
        else:
            if coord_x > valor_limite:
                resultado.append(p)
    return resultado

def obtener_franja_cercana(lista_puntos, centro_x, distancia_minima):
    # Obener puntos dentro de la franja en referencia al codigo en
    ↪ ClosessPair_Code.cpp sobre "vector<Point> stripPoints;"
    franja = []
    for p in lista_puntos:
        coord_x = obtener_coordenada_eje(p, 0)
        diferencia_x = abs(coord_x - centro_x)
        if diferencia_x < distancia_minima:
            franja.append(p)
    return franja

def closest_pair_brute_force(puntos):
    # fuerza bruta en referencia al codigo en ClosessPair_Code.cpp sobre
    ↪ "vector<Point> bruteForce(vector<Point> p)"
    # asi mismo en revision de la paper paper3-Parallelizable-Algorithm
    ↪ "Algorithm 2: Brute-force 1-NN similarity join."
    n = len(puntos)
    if n < 2:
        return float('inf'), (None, None)

    distancia_minima = float('inf')
    mejor_par = (puntos[0], puntos[1])

    for i in range(n):
        for j in range(i + 1, n):
            d = calcular_distancia(puntos[i], puntos[j])
            if d < distancia_minima:
                distancia_minima = d
                mejor_par = (puntos[i], puntos[j])

    return distancia_minima, mejor_par

def closest_pair_1d_sorted(puntos_ordenados):

```

```

    # Resolver 1D considerando que la lista se encuentra ordenada, los 2 puntos
    ↪ mas cercanos.
    distancia_minima = float('inf')
    mejor_par = (None, None)

    n = len(puntos_ordenados)
    for i in range(n - 1):
        distancia = puntos_ordenados[i+1] - puntos_ordenados[i]
        if distancia < distancia_minima:
            distancia_minima = distancia
            mejor_par = (puntos_ordenados[i], puntos_ordenados[i+1])

    return distancia_minima, mejor_par

def _closest_pair_2d_rec(px, py):
    # Algoritmo recursivo divide y venceras para 2D
    # Basado en el codigo ClosessPair_Code.cpp sobre la funcion "vector<Point>
    ↪ closestPair(vector<Point> points)"
    n = len(px)
    if n <= 3:
        return closest_pair_brute_force(px)

    mitad = n // 2
    punto_medio = px[mitad]
    limite_x = punto_medio[0]

    py_izquierda = filtrar_puntos_por_limite_x(py, limite_x,
    ↪ es_menor_igual=True)
    py_derecha = filtrar_puntos_por_limite_x(py, limite_x, es_menor_igual=False)

    dist_izq, par_izq = _closest_pair_2d_rec(px[:mitad], py_izquierda)
    dist_der, par_der = _closest_pair_2d_rec(px[mitad:], py_derecha)

    if dist_izq < dist_der:
        d_min = dist_izq
        mejor_par = par_izq
    else:
        d_min = dist_der
        mejor_par = par_der

    # Creacion de franja, basado en el codigo ClosessPair_Code.cpp sobre
    ↪ "vector<Point> stripPoints;"
    franja = obtener_franja_cercana(py, limite_x, d_min)

    tamano_franja = len(franja)
    for i in range(tamano_franja):
        j = i + 1

```

```

        while j < tamano_franja:
            diferencia_y = franja[j][1] - franja[i][1]
            if diferencia_y >= d_min:
                break

            d = calcular_distancia(franja[i], franja[j])
            if d < d_min:
                d_min = d
                mejor_par = (franja[i], franja[j])
            j = j + 1

    return d_min, mejor_par

def closest_pair_2d(puntos):
    # Llamar a divide y venceras en 2D, basado en el codigo ClosessPair_Code.
    ↪cpp entre las lineas 313-315
    px = ordenar_puntos_por_eje(puntos, 0) # Ordenar por X
    py = ordenar_puntos_por_eje(puntos, 1) # Ordenar por Y
    return _closest_pair_2d_rec(px, py)

def _closest_pair_3d_rec(px):
    # Basado en el paper "Algorithm 3 (sección Partition Strategy)"
    n = len(px)
    if n <= 3:
        return closest_pair_brute_force(px)

    mitad = n // 2
    punto_medio = px[mitad]
    limite_x = punto_medio[0]

    dist_izq, par_izq = _closest_pair_3d_rec(px[:mitad])
    dist_der, par_der = _closest_pair_3d_rec(px[mitad:])

    if dist_izq < dist_der:
        d_min = dist_izq
        mejor_par = par_izq
    else:
        d_min = dist_der
        mejor_par = par_der

    franja = obtener_franja_cercana(px, limite_x, d_min)
    franja_ordenada_y = ordenar_puntos_por_eje(franja, 1)

    tamano_franja = len(franja_ordenada_y)
    for i in range(tamano_franja):
        j = i + 1
        while j < tamano_franja:

```

```

        diferencia_y = franja_ordenada_y[j][1] - franja_ordenada_y[i][1]
        if diferencia_y >= d_min:
            break

        diferencia_z = abs(franja_ordenada_y[j][2] -
        ↪franja_ordenada_y[i][2])
        if diferencia_z < d_min:
            d = calcular_distancia(franja_ordenada_y[i],
        ↪franja_ordenada_y[j])
            if d < d_min:
                d_min = d
                mejor_par = (franja_ordenada_y[i], franja_ordenada_y[j])
            j = j + 1

    return d_min, mejor_par

def closest_pair_3d(puntos):
    # Llamar al algoritmo divide y venceras en 3D
    px = ordenar_puntos_por_eje(puntos, 0)
    return _closest_pair_3d_rec(px)

```

```

[5]: # =====
# PRUEBAS
# =====

if __name__ == "__main__":
    print("=====")
    print(" DEMOSTRACIÓN ")
    print("=====")

    # 1. Ejemplo 1D
    pts_1d = [3403.0, 4537.0, 9089.0, 9746.0, 7259.0, 4540.0]
    pts_1d_ordenados = sorted(pts_1d) # Ordenamiento basico incorporado
    dist_bf_1d, par_bf_1d = closest_pair_brute_force(pts_1d)
    dist_dv_1d, par_dv_1d = closest_pair_1d_sorted(pts_1d_ordenados)

    print("\n--- PRUEBAS EN 1D ---")
    print("Puntos originales:", pts_1d)
    print("Fuerza Bruta: Distancia =", dist_bf_1d, "| Par =", par_bf_1d)
    print("Divide y Vencerás: Distancia =", dist_dv_1d, "| Par =", par_dv_1d)

    # 2. Ejemplo 2D
    pts_2d = [(1122.0, 6175.0), (135.0, 4076.0), (7296.0, 2741.0), (1130.0,
    ↪6180.0), (5000.0, 5000.0)]
    dist_bf_2d, par_bf_2d = closest_pair_brute_force(pts_2d)
    dist_dv_2d, par_dv_2d = closest_pair_2d(pts_2d)

```

```

print("\n--- PRUEBAS EN 2D ---")
print("Puntos originales:", pts_2d)
print("Fuerza Bruta:      Distancia =", round(dist_bf_2d, 2), "| Par =",
↪par_bf_2d)
print("Divide y Vencerás: Distancia =", round(dist_dv_2d, 2), "| Par =",
↪par_dv_2d)

# 3. Ejemplo 3D
pts_3d = [(1.0, 2.0, 3.0), (10.0, 20.0, 30.0), (1.5, 2.5, 3.5), (100.0, 200.
↪0, 300.0), (2.0, 2.0, 2.0)]
dist_bf_3d, par_bf_3d = closest_pair_brute_force(pts_3d)
dist_dv_3d, par_dv_3d = closest_pair_3d(pts_3d)

print("\n--- PRUEBAS EN 3D ---")
print("Puntos originales:", pts_3d)
print("Fuerza Bruta:      Distancia =", round(dist_bf_3d, 2), "| Par =",
↪par_bf_3d)
print("Divide y Vencerás: Distancia =", round(dist_dv_3d, 2), "| Par =",
↪par_dv_3d)

```

DEMOSTRACIÓN

--- PRUEBAS EN 1D ---

Puntos originales: [3403.0, 4537.0, 9089.0, 9746.0, 7259.0, 4540.0]

Fuerza Bruta: Distancia = 3.0 | Par = (4537.0, 4540.0)

Divide y Vencerás: Distancia = 3.0 | Par = (4537.0, 4540.0)

--- PRUEBAS EN 2D ---

Puntos originales: [(1122.0, 6175.0), (135.0, 4076.0), (7296.0, 2741.0),
(1130.0, 6180.0), (5000.0, 5000.0)]

Fuerza Bruta: Distancia = 9.43 | Par = ((1122.0, 6175.0), (1130.0, 6180.0))

Divide y Vencerás: Distancia = 9.43 | Par = ((1122.0, 6175.0), (1130.0, 6180.0))

--- PRUEBAS EN 3D ---

Puntos originales: [(1.0, 2.0, 3.0), (10.0, 20.0, 30.0), (1.5, 2.5, 3.5),
(100.0, 200.0, 300.0), (2.0, 2.0, 2.0)]

Fuerza Bruta: Distancia = 0.87 | Par = ((1.0, 2.0, 3.0), (1.5, 2.5, 3.5))

Divide y Vencerás: Distancia = 0.87 | Par = ((1.0, 2.0, 3.0), (1.5, 2.5, 3.5))